



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251176
Nama Lengkap	David Alexsi Cahyadi
Minggu ke / Materi	07 / Percabangan dan Perulangan Kompleks

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

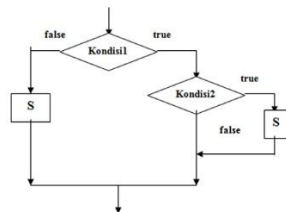
BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Struktur Percabangan Kompleks

Struktur percabangan kompleks adalah kondisi di mana kondisi pemilihan tidak hanya satu, tetapi bisa terdiri dari banyak alternatif kondisi. Pada percabangan kompleks, perintah yang dikerjakan juga bisa lebih dari satu dan biasanya menggunakan **IF** bertingkat atau kombinasi **IF**, **ELIF**, dan **ELSE**. Struktur ini digunakan ketika suatu program memiliki banyak kemungkinan kondisi yang harus diperiksa.

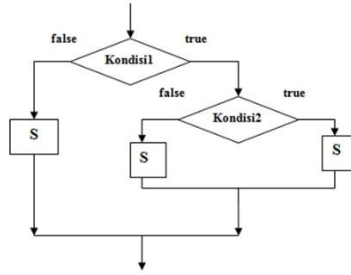
Percabangan kompleks memiliki beberapa bentuk struktur IF bertingkat. Bentuk-bentuk ini menunjukkan bagaimana IF dapat berada di dalam IF lainnya atau digabungkan dengan ELSE dan ELIF untuk membuat keputusan yang lebih kompleks. Setiap bentuk percabangan kompleks memiliki flowchart yang berbeda sesuai dengan alur logika program



Gambar 6.1: Flowchart Percabangan Kompleks Bentuk 1, Sumber:

<https://shorturl.at/z066V>

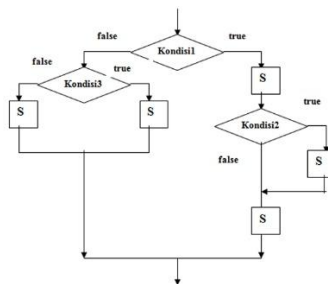
Pada percabangan kompleks bentuk pertama, terdapat IF di dalam IF, sehingga jika kondisi pertama terpenuhi maka program akan memeriksa kondisi kedua. Jika kondisi kedua terpenuhi maka instruksi dijalankan, jika tidak maka instruksi pada ELSE dijalankan.



Gambar 6.2: Flowchart Percabangan Kompleks Bentuk 2, Sumber:

<https://shorturl.at/z066V>

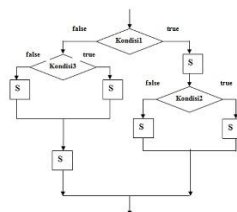
Pada percabangan kompleks bentuk kedua, terdapat IF di dalam IF dan juga ELSE pada IF pertama. Artinya, jika kondisi pertama tidak terpenuhi maka program langsung menjalankan perintah pada ELSE tanpa memeriksa kondisi kedua.



Gambar 6.3: Flowchart Percabangan Kompleks Bentuk 3, Sumber:

<https://shorturl.at/z066V>

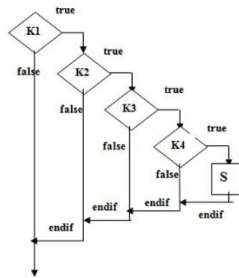
Pada percabangan kompleks bentuk ketiga, terdapat beberapa IF yang disusun bertingkat dengan ELSE yang juga berisi IF lain. Struktur ini digunakan ketika terdapat banyak kondisi yang saling berkaitan.



Gambar 6.4: Flowchart Percabangan Kompleks Bentuk 4, Sumber:

<https://shorturl.at/z066V>

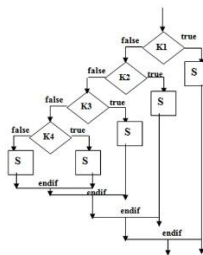
Pada percabangan kompleks bentuk keempat, struktur IF dan ELSE lebih kompleks lagi karena pada bagian ELSE masih terdapat IF dan ELSE lainnya. Struktur ini biasanya digunakan pada kasus yang memiliki banyak kemungkinan kondisi.



Gambar 6.5: Flowchart Percabangan Kompleks Bentuk 5, Sumber:

<https://shorturl.at/z066V>

Pada percabangan kompleks bentuk kelima, terdapat IF bertingkat lebih dari dua level. Struktur ini biasanya digunakan untuk memeriksa banyak kondisi yang harus terpenuhi secara berurutan.



Gambar 6.6: Flowchart Percabangan Kompleks Bentuk 6, Sumber:

<https://shorturl.at/z066V>

Pada percabangan kompleks bentuk keenam, struktur IF dan ELSE bertingkat digunakan untuk memilih satu kondisi dari banyak kondisi yang ada secara berurutan dari atas ke bawah.

Penggunaan IF bertingkat memiliki keuntungan yaitu eksekusi program menjadi lebih efisien. Hal ini karena tidak semua kondisi akan diperiksa, hanya kondisi yang diperlukan saja. Berbeda dengan IF biasa yang semua kondisi akan diperiksa satu per satu walaupun pada akhirnya hanya satu kondisi yang terpenuhi. Contoh kasus yang cocok menggunakan IF bertingkat adalah konversi nilai yang telah ditentukan.

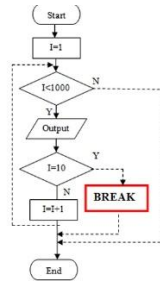
Struktur Perulangan Kompleks

Struktur perulangan kompleks adalah struktur perulangan yang lebih kompleks dibandingkan perulangan biasa. Perulangan kompleks meliputi penggunaan perintah **break**, **continue**, dan **perulangan bertingkat (nested loop)**.

Break

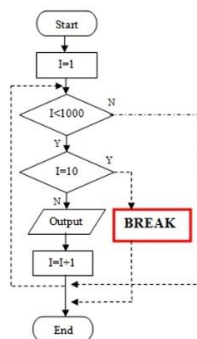
Perintah break digunakan untuk menghentikan proses perulangan yang sedang berjalan sebelum perulangan selesai. Biasanya perintah break digunakan bersama dengan kondisi if. Jika kondisi tertentu terpenuhi, maka perulangan akan langsung dihentikan.

Sebagai contoh, jika sebuah perulangan berjalan dari 0 sampai 1000 tetapi terdapat perintah break ketika nilai mencapai 10, maka perulangan akan berhenti pada angka 10 walaupun batas perulangan masih jauh.



Gambar 6.7: Flowchart Program Menggunakan Break, Sumber: <https://shorturl.at/z066V>

Jika perintah **print** diletakkan sebelum **break**, maka nilai yang menyebabkan **break** masih akan ditampilkan. Namun jika **print** diletakkan setelah **break**, maka nilai tersebut tidak akan ditampilkan karena perulangan sudah dihentikan terlebih dahulu.

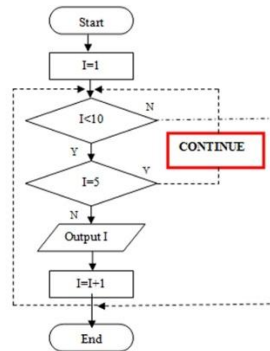


Gambar 6.8: Flowchart Program Menggunakan Break 2, Sumber: <https://shorturl.at/z066V>

Continue

Perintah **continue** digunakan untuk melewati satu iterasi perulangan dan langsung melanjutkan ke iterasi berikutnya. Perintah setelah **continue** tidak akan dijalankan pada iterasi tersebut.

Sebagai contoh, jika terdapat perulangan dari 0 sampai 9 dan ketika angka 5 dijalankan perintah continue, maka angka 5 tidak akan ditampilkan karena program langsung melanjutkan ke iterasi berikutnya.



Gambar 6.9: Flowchart Program Menggunakan Continue, Sumber:

<https://shorturl.at/z066V>

Perulangan bertingkat (Nested Loop)

Perulangan bertingkat (Nested Loop) adalah perulangan yang berada di dalam perulangan lain. Pada perulangan bertingkat terdapat dua bagian yaitu **outer loop (perulangan luar)** dan **inner loop (perulangan dalam)**. Cara kerjanya adalah setiap satu kali outer loop berjalan, maka inner loop akan dijalankan sesuai dengan. Jika outer loop berjalan sebanyak m kali dan inner loop berjalan sebanyak n kali, total perulangan.

Perulangan bertingkat biasanya digunakan untuk menyelesaikan masalah yang memiliki pola terbentuk tabel atau grid seperti matriks, pola bintang, dan game board. Struktur perulangan bertingkat dapat menggunakan **for di dalam for** atau **while di dalam while**, di mana perulangan luar biasanya mengatur baris dan perulangan dalam mengatur kolom.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github:https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 7.1

Soal:

Buatlah program untuk mencari bilangan prima terdekat dari suatu bilangan yang diinputkan oleh pengguna (n) dan nilai bilangan prima tersebut $< n$.

Contoh:

input n=12, maka prima terdekat < 12 adalah 11

Contoh:

input n=21, maka prima terdekat < 21 adalah 19.

Source Code:

```
#Latihan 7.1
def prima (n):
    if n < 2:
        return False
    for i in range (2, n):
        if n % i == 0:
            return False
    return True

n = int(input("Masukkan angka n: "))

for i in range(n-1, 1 ,-1):
    if prima(i):
        print("Prima terdekat <", n, "adalah", i)
        break
```

Output:

```
PS D:\PrAlpro\Pertemuan 7> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 7/.dist/soal01.py"
Prima terdekat < 33 adalah 31
PS D:\PrAlpro\Pertemuan 7> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 7/.dist/soal01.py"
Masukkan angka n: 38
Prima terdekat < 38 adalah 37
```

Penjelasan:

Program mendefinisikan fungsi **prima(n)** yang berfungsi untuk menentukan apakah suatu bilangan termasuk bilangan prima atau bukan. Di dalam fungsi tersebut, jika nilai n kurang dari 2 maka langsung mengembalikan nilai **False** karena bilangan prima harus lebih besar atau sama dengan 2. Kemudian program melakukan perulangan dari angka 2 sampai $n-1$ untuk mengecek apakah n habis dibagi oleh salah satu angka tersebut. Jika ditemukan bahwa n habis dibagi ($n \% i == 0$), maka fungsi mengembalikan **False** karena berarti n bukan bilangan prima. Jika tidak ada angka yang bisa membagi n sampai perulangan selesai, maka fungsi mengembalikan **True** yang berarti n adalah bilangan prima.

Setelah fungsi prima dibuat, program meminta pengguna memasukkan sebuah bilangan n . Kemudian program melakukan perulangan mundur menggunakan **for i in range(n-1, 1, -1)** yang artinya program akan mengecek angka dari $n-1$ turun sampai 2. Pada setiap perulangan, program memanggil fungsi **prima(i)** untuk mengecek apakah angka tersebut adalah bilangan prima. Jika ditemukan bilangan prima, maka program akan menampilkan bahwa bilangan tersebut adalah bilangan prima terdekat yang lebih kecil dari n , lalu perulangan dihentikan dengan perintah **break** sehingga program tidak melanjutkan pengecekan angka berikutnya.

Latihan 7.2

Soal:

Buatlah program untuk menampilkan deret seperti di bawah ini. n diinputkan secara dinamis

```
contoh: n = 6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

Source Code:

```
#Latihan 7.2
import math

n = int(input("Masukkan angka n: "))

for i in range (n, 0, -1):
    faktor = math.factorial(i)

    print(faktor, end=" ")

    for j in range(i, 0, -1):
        print(j, end=" ")
    print()
```

Output:

```
PS D:\PrAlpro\Pertemuan 7> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 7/.dist/soal02.py"
Masukkan angka n: 8
40320 8 7 6 5 4 3 2 1
5040 7 6 5 4 3 2 1
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

Penjelasan:

Program meminta input nilai n dari pengguna. Kemudian perulangan luar (outer loop) menggunakan **for i in range (n, 0, -1)** yang berfungsi untuk mengulang dari n sampai 1, dimana setiap nilai i akan menjadi jumlah bilangan yang ditampilkan pada baris tersebut. Di dalam perulangan luar, program menghitung nilai faktorial dari i menggunakan fungsi **math.factorial(i)** lalu mencetak hasil faktorial tersebut terlebih dahulu.

Selanjutnya terdapat perulangan dalam (inner loop) yaitu **for j in range(l, 0, -1)** yang berfungsi untuk menampilkan deret angka dari i turun sampai 1 pada baris yang sama. Setelah inner loop selesai, perintah **print()** digunakan untuk dipindahkan ke baris berikutnya.

Latihan 7.3

Soal:

Buatlah program untuk menampilkan deret seperti di bawah ini. n diinputkan secara dinamis

contoh: tinggi = 5, lebar = 4

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
```

Source Code:

```
#Latihan 7.3
t = int(input("Masukkan tinggi: "))
l = int(input("Masukkan lebar: "))

kosong = 1

for i in range (1, t +1):
    for i in range (1, l + 1):
        print (kosong, end=' ')
        kosong += 1
    print()
```

Output:

```
PS D:\PrAlpro\Pertemuan 7> & C:\Users\dvdal\AppData\Local\Programs\Python\Python313\python.exe "d:/PrAlpro/Pertemuan 7/.dist/soal03.py"
Masukkan tinggi: 6
Masukkan lebar: 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
21 22 23 24
```

Penjelasan:

Program dimulai dengan meminta pengguna untuk memasukkan tinggi dan lebar yang diinginkan. Kemudian dibuat variabel **kosong** yang bernilai awal 1 dan akan terus bertambah setiap kali angka ditampilkan. Program menggunakan perulangan bertingkat, dimana perulangan luar digunakan untuk mengatur jumlah baris dari 1 sampai tinggi, sedangkan perulangan dalam digunakan untuk mencetak angka sebanyak lebar pada setiap baris. Di dalam inner loop, program mencetak nilai **kosong**, lalu nilai tersebut ditambah 1 agar berikutnya bertambah secara berurutan. Setelah inner loop selesai, program mencetak baris baru menggunakan **print()**.